Documentación completa 2.4 Dependencias

Backend City Care | Versión 1.0

Resultado. El apartado 2.4 está terminado. City Care permite consultar, crear, editar, desactivar y reactivar dependencias; asignar responsables; proteger sus usuarios ante eliminaciones; y consumir todas estas operaciones mediante OpenAPI y Orval.

Estado	Cobertura	Evidencia
COMPLETO	6 de 6 puntos del apartado 2.4	13 pruebas funcionales del módulo
VALIDADO	Sin regresiones en 2.1, 2.2 y 2.3	46 de 46 pruebas aprobadas
SINCRONIZADO	MySQL, OpenAPI y Orval	7 migraciones y 16 rutas API

1 Alcance terminado

Punto	Implementación	Estado
Tabla y relación	dependencies relacionada con users mediante dependency_id	Listo
Dependencia inicial	El seeder conserva la dependencia administrativa base	Listo
Listado	Paginación, búsqueda, estado y responsables	Listo
CRUD	Alta, detalle, edición y baja lógica	Listo
Integridad	ON DELETE RESTRICT evita borrados en cascada	Listo
Responsables	Asignación transaccional de dependency_admin	Listo

2 Modelo de datos

Campo	Tipo	Regla
id	unsigned integer	Llave primaria
name	varchar	Obligatorio y único mediante uq_dependencies_name
description	text nullable	Descripción administrativa opcional
is_active	boolean	Baja lógica; valor inicial true
created_at / updated_at	datetime	Trazabilidad temporal

La migración 1793479000000_secure_and_activate_dependencies añade el estado, el índice único y reemplaza la clave foránea anterior. La relación users.dependency_id ahora aplica ON DELETE RESTRICT y ON UPDATE CASCADE.

```
users.dependency_id -> dependencies.id
ON DELETE RESTRICT
ON UPDATE CASCADE
```

3 Contrato HTTP

Método	Ruta	Operación	Permiso
GET	/api/v1/admin/dependencies	Listar, buscar y filtrar	dependencies.list
POST	/api/v1/admin/dependencies	Crear	dependencies.create
GET	/api/v1/admin/dependencies/:id	Consultar detalle	dependencies.read
PATCH	/api/v1/admin/dependencies/:id	Editar nombre o descripción	dependencies.update
PATCH	/api/v1/admin/dependencies/:id/status	Activar o desactivar	dependencies.status
PATCH	/api/v1/admin/dependencies/:id/managers	Reemplazar responsables	dependencies.manage

3.1 Listado

- Acepta page, perPage, search e isActive.
- Incluye id, nombre, descripción, estado y responsables activos.
- El administrador general ve todas; dependency_admin solamente su dependencia.

3.2 Alta y edición

- El nombre es obligatorio al crear y no puede repetirse.
- La edición parcial exige al menos un campo.
- No se expone DELETE físico; el estado se modifica con la ruta /status.
- No se puede crear ni trasladar un usuario a una dependencia inactiva.

3.3 Baja lógica

La desactivación se rechaza con conflicto 409 mientras exista al menos un usuario activo. Esto evita dejar cuentas operativas dentro de una dependencia cerrada. Una dependencia vacía puede desactivarse y reactivarse sin perder su historial.

4 Responsables por dependencia

Los responsables son usuarios activos de la misma dependencia cuyo rol es dependency_admin. La ruta de responsables reemplaza el conjunto completo dentro de una transacción.

Validación	Comportamiento
Misma dependencia	No admite usuarios pertenecientes a otra dependencia
Cuenta activa	No admite responsables inactivos

Validación	Comportamiento
Sin duplicados	Cada identificador aparece una sola vez
Jerarquía	No admite cuentas con rol admin
Promoción	Los seleccionados cambian a dependency_admin
Reemplazo	Los responsables retirados regresan al rol user

5 Autorización

Acción	admin	dependency_admin	user
Listar dependencias	Todas	Solo la propia	No
Consultar detalle	Cualquiera	Solo la propia	No
Crear	Sí	No	No
Editar	Sí	No	No
Cambiar estado	Sí	No	No
Asignar responsables	Sí	No	No

Cada ruta declara un permiso exacto mediante el middleware parametrizado. El alcance de dependency_admin se valida adicionalmente por dependency_id; conocer otro identificador no permite consultar ni administrar otra dependencia.

6 Respuestas y errores

Código	Identificador	Significado
401	E_UNAUTHORIZED_ACCESS	Falta token o no es válido
403	E_PERMISSION_REQUIRED	El rol no posee el permiso
403	E_DEPENDENCY_SCOPE	Intento de acceso fuera del alcance
404	E_DEPENDENCY_NOT_FOUND	La dependencia no existe
409	E_DEPENDENCY_HAS_ACTIVE_USERS	No puede desactivarse con usuarios activos
409	E_DEPENDENCY_INACTIVE	No puede recibir usuarios
422	E_EMPTY_DEPENDENCY_UPDATE	Edición sin campos
422	E_INVALID_DEPENDENCY_MANAGERS	Selección de responsables inválida
422	E_VALIDATION_ERROR	Datos o nombre duplicado inválidos

7 Auditoría

- dependency_created registra actor y dependencia creada.
- dependency_updated registra actor, dependencia y campos modificados.
- dependency_status_updated registra actor, dependencia y estado.
- dependency_managers_updated registra actor, dependencia y responsables finales.

8 OpenAPI y cliente Orval

El Swagger fue exportado desde el backend y Orval regeneró el SDK TypeScript. Las funciones disponibles para el futuro frontend son:

Función	Uso
listDependencies	Listado paginado y filtros
createDependency	Alta administrativa
getDependency	Consulta por id
updateDependency	Edición parcial
updateDependencyStatus	Baja lógica o reactivación
updateDependencyManagers	Reemplazo de responsables

```
npm run openapi:generate
# salida: sdk/generated/city-care.ts
```

9 Verificación automatizada

Prueba del 2.4	Resultado
Autenticación y permiso requerido	Aprobada
Listado, búsqueda, filtro y responsables	Aprobada
Alcance del administrador de dependencia	Aprobada
Alta y nombre único	Aprobada
Dependencia inactiva no recibe usuarios	Aprobada
Edición parcial y cuerpo vacío	Aprobada
Baja lógica con protección de usuarios	Aprobada
Asignación y reemplazo transaccional	Aprobada
Responsables externos, inactivos o superiores	Aprobada
Integridad RESTRICT y ausencia de DELETE	Aprobada

Comprobación	Resultado

Comprobación	Resultado
npm run typecheck	Sin errores
npm run lint	Sin errores
npm run build	Compilación exitosa
npm test	46 pruebas aprobadas
OpenAPI + Orval	Generación exitosa

10 Archivos principales

Archivo	Responsabilidad
app/controllers/dependencies_controller.ts	Casos de uso HTTP
app/validators/dependency.ts	Validación de entradas
app/services/authorization_service.ts	Permisos, alcance y estado
database/migrations/1793479000000_secure_and_activate_dependencies.ts	Estado e integridad
tests/functional/dependencies.spec.ts	Cobertura funcional
openapi/city-care.v1.yaml	Contrato OpenAPI
sdk/generated/city-care.ts	Cliente Orval generado

11 Conclusión

El apartado 2.4 queda cerrado: datos protegidos, administración completa con baja lógica, responsables con alcance controlado, contrato sincronizado y pruebas aprobadas. El siguiente bloque de trabajo es 2.5: integración final del SDK, Bearer y URL por entorno en el frontend.